

Lesson 5

Local State, Controlled Components & Forms

Code the Dream — React

Today's Session

EXPLORE

- Conditional rendering: ternary, &&, and functions
- Local state vs. application state
- Controlled vs. uncontrolled forms
- The controlled component update cycle
- Managing multiple inputs
- Validation with `touched` state

APPLY

- Convert an uncontrolled form to controlled
- Add `touched-aware` validation with inline error messages
- Update form submission to use state values

By the end: show and hide UI based on state, manage form inputs with React state, and validate fields in a user-friendly way.

Conditional Rendering — Ternary

The **ternary operator** renders one thing or another based on a condition.

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Use `null` on one side to show or hide a single element:

```
const [isOpen, setIsOpen] = useState(false);
```

```
{isOpen ? <Modal /> : null}  
<button onClick={() => setIsOpen(!isOpen)}>Toggle</button>
```

Ternary is the right choice when you need **either/or** — two different things depending on a condition.

Conditional Rendering — &&

The **logical && operator** shows something or nothing.

```
{hasError && <p className="error">{errorMessage}</p>}
```

```
{isCartOpen && <Cart cart={cart} handleClose={handleCloseCart} />}
```

JavaScript evaluates the left side first. If it's falsy, the right side is never evaluated — nothing renders.

Watch out: if the left side is `0`, React renders the number `0` instead of nothing. Use a boolean: `{items.length > 0 && ...}`

Choosing Between Ternary and &&

Situation	Best choice
Show A or show B	Ternary: condition ? <A /> :
Show A or show nothing	&&: condition && <A />
More than two outcomes	Function with if/switch inside the component

```
// Function-based – cleaner for complex logic
function renderStatus(status) {
  if (status === 'loading') return <Spinner />;
  if (status === 'error') return <ErrorMessage />;
  return <Content />;
}

return <div>{renderStatus(status)}</div>;
```

Check for Understanding

The `cart` array has 0 items. Which renders something unexpected?

```
// A  
{cart.length > 0 && <CartList />}
```

```
// B  
{cart.length && <CartList />}
```

```
// C  
{cart.length === 0 ? <p>Empty</p> : <CartList />}
```

```
// D  
{isCartOpen && <CartList />}
```

A) Option A B) Option B C) Option C D) Option D

Local State vs. Application State

Not all state belongs at the top of the component tree.

Application state — lives in a top-level component, shared across the app

- Todo list
- Shopping cart contents
- Logged-in user

Local state — lives close to where it's used, not shared

- What a user is currently typing in a form
- Whether a dropdown is open
- A working copy of data being edited

Keep form input state **local** to the form component. Only push it up to application state when the user confirms (submits).

Controlled vs. Uncontrolled Forms

Uncontrolled — DOM manages the value

```
const inputRef = useRef();

function handleSubmit(e) {
  e.preventDefault();
  console.log(inputRef.current.value);
}

<input ref={inputRef} />
```

- Less code to write
- Good for simple forms

Controlled — React state manages the value

```
const [name, setName] = useState('');

function handleSubmit(e) {
  e.preventDefault();
  console.log(name);
}

<input value={name}
  onChange={e => setName(e.target.value)} />
```

- Real-time validation
- Easier to sync with other state

The Controlled Component Update Cycle

Every keystroke triggers a loop:

1. User types → `onChange` fires
2. Handler calls state setter with the new value
3. React re-renders the component
4. Input's `value` prop reflects the new state

```
const [title, setTitle] = useState('');
```

```
<input  
  value={title}  
  onChange={e => setTitle(e.target.value)}  
>
```

This happens in milliseconds — the input feels instant to the user, but React controls every character.

The `value` prop is what makes it controlled. Without it, the input is uncontrolled even if you have an `onChange`.

Managing Multiple Inputs

Instead of one `useState` per field, use an object and a single handler.

```
const [fields, setFields] = useState({ name: '', rating: '' });

function handleChange(e) {
  const { name, value } = e.target;
  setFields(prev => ({ ...prev, [name]: value }));
}

<input name="name" value={fields.name} onChange={handleChange} />
<input name="rating" value={fields.rating} onChange={handleChange} />
```

`e.target.name` matches the input's `name` attribute — the handler uses it as the key to update only that field.

Validation with touched State

Show errors only **after** a user has interacted with a field — not on first load.

```
const [touched, setTouched] = useState({ name: false, rating: false });
```

```
function validateName() { return name.trim() !== ''; }
```

```
function getNameError() {  
  if (touched.name && !validateName()) return 'Name is required';  
  return null;  
}
```

```
<input  
  value={name}  
  onChange={e => setName(e.target.value)}  
  onFocus={() => setTouched(prev => ({ ...prev, name: true })))}  
>  
{getNameError() && <p className="error">{getNameError()}</p>}
```

Check for Understanding

A form has a controlled input. The user hasn't clicked on the name field yet. `touched.name` is `false`. The name value is empty. What appears on screen?

- A) An error message: "Name is required"
- B) Nothing — the error only shows when `touched.name` is `true`
- C) The input is disabled until the user submits
- D) The form submits automatically because the field is empty

Now: Apply

Let's convert a form from uncontrolled to controlled.

This Week's Exercise — Snack Manager

You'll work inside `src/exercises/lesson-05/SnackForm.jsx`.

The form currently uses `FormData` to read values on submit (uncontrolled). Your job:

1. **Convert to controlled** — add state variables, wire up `value` and `onChange`
2. **Add validation** — validate fields with `touched` state, show inline errors
3. **Update submission** — use state values instead of `FormData`, prevent invalid submits

Read through `SnackForm.jsx` and `SnackManager.jsx` before making changes. Understand what already works before you change it.

Step 1 — Add State Variables

At the top of `SnackForm`, replace `defaultValue` inputs with three state variables:

```
const [name, setName] = useState('');
const [rating, setRating] = useState('');
const [touched, setTouched] = useState({ name: false, rating: false });
```

Then add a `useEffect` to populate the form when editing an existing snack:

```
useEffect(() => {
  if (editingSnack) {
    setName(editingSnack.name);
    setRating(editingSnack.rating);
  } else {
    setName('');
    setRating('');
  }
  setTouched({ name: false, rating: false });
}, [editingSnack]);
```

Step 2 — Convert Inputs to Controlled

Update each input with `value`, `onChange`, and `onFocus`:

```
<input
  type="text"
  value={name}
  onChange={e => setName(e.target.value)}
  onFocus={() => setTouched(prev => ({ ...prev, name: true })))}
/>

<input
  type="number"
  min="1" max="5"
  value={rating}
  onChange={e => setRating(e.target.value)}
  onFocus={() => setTouched(prev => ({ ...prev, rating: true })))}
/>
```

Remove the `required` attribute — React will handle validation instead.

Step 3 — Add Validation & Error Display

Write validation functions and display errors below each input:

```
function validateName() { return name.trim() !== ''; }
function validateRating() { return rating !== ''; }

function getNameError() {
  return touched.name && !validateName() ? 'Snack name is required' : null;
}
function getRatingError() {
  return touched.rating && !validateRating() ? 'Please select a rating' : null;
}

{getNameError() && <div className={styles.error}>{getNameError()}</div>}
{getRatingError() && <div className={styles.error}>{getRatingError()}</div>}
```

Step 4 — Update Form Submission

Replace `FormData` extraction with state values, and block invalid submits:

```
function handleSubmit(e) {
  e.preventDefault();

  if (!validateName() || !validateRating()) {
    setTouched({ name: true, rating: true }); // reveal all errors
    return;
  }

  onSubmit({ name, rating: Number(rating) });

  if (!editingSnack) {
    setName('');
    setRating('');
    setTouched({ name: false, rating: false });
  }
}
```

Wrap-Up

This week you learned:

- Ternary renders A or B; `&&` renders something or nothing — choose based on the situation
- Keep form state local to the form; only push to app state on submit
- A controlled input pairs `value` (state) with `onChange` (setter) — both are required
- `touched` state delays validation errors until after the user interacts with a field
- One `handleChange` with `e.target.name` can manage all fields in a form

Before next session: Complete the weekly assignment — add conditional rendering for an empty todo list, convert `TodoForm` to a controlled component, and add a checkbox to complete todos. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.